

SM – Assignment 01**Name:** Hafsa Salman**Roll no.** 22K-5161**Question no. 01**

```
import numpy as np

#Rounds: 100000
N_ROUNDS = 100000
results = []

for rnd in range(N_ROUNDS):
    prob_patent = np.random.randint(0, 2)
    base_sales = round(np.random.triangular(1e6, 3e6, 9e6) / 1e6, 2)
    patent_markup = np.random.triangular(0.25, 0.5, 0.75)
    final_sales = round(base_sales + (base_sales * prob_patent * patent_markup), 2)

    results.append(final_sales)

#Average Expected Sales
average_sales = np.mean(results)
#Probability of selling it more than $6M
prob_above_6M = np.sum(np.array(results) > 6) / N_ROUNDS
#Probability of selling it more than $11M
prob_above_11M = np.sum(np.array(results) > 11) / N_ROUNDS
#Results
print(f"Expected Average Sales: ${average_sales:.2f}M")
print(f"Probability of sales greater $6M: {prob_above_6M:.2%}")
print(f"Probability of sales greater $11M: {prob_above_11M:.2%}")
#Is it a good deal?
if average_sales > 6:
    print("Not a good deal! The sales are higher than $6M.")
else:
    print("Good deal! The sales are lower than $6M.")
```

✓ 3.2s

```
Expected Average Sales: $5.41M
Probability of sales greater $6M: 35.08%
Probability of sales greater $11M: 2.99%
Good deal! The sales are lower than $6M.
```

Explanation:

We were asked to run it 100000 times. I have done it in the code as well as the excel sheet.

Sir gave the code with the formulae. I completed it answering all the questions asked in the PDF.

The same block of code has been used for Question no. 02 (PDF) but for 10000 and 1000 rounds.

Question no. 02 (PDF)

//For 10000 rounds

```
import numpy as np

#Rounds: 10000
N_ROUNDS = 10000
results = []

for rnd in range(N_ROUNDS):
    prob_patent = np.random.randint(0, 2)
    base_sales = round(np.random.triangular(1e6, 3e6, 9e6) / 1e6, 2)
    patent_markup = np.random.triangular(0.25, 0.5, 0.75)
    final_sales = round(base_sales + (base_sales * prob_patent * patent_markup), 2)

    results.append(final_sales)

#Average Expected Sales
average_sales = np.mean(results)
#Probability of selling it more than $6M
prob_above_6M = np.sum(np.array(results) > 6) / N_ROUNDS
#Probability of selling it more than $11M
prob_above_11M = np.sum(np.array(results) > 11) / N_ROUNDS
#Results
print(f"Expected Average Sales: ${average_sales:.2f}M")
print(f"Probability of sales greater than $6M: {prob_above_6M:.2%}")
print(f"Probability of sales greater than $11M: {prob_above_11M:.2%}")
#Is it a good deal?
if average_sales > 6:
    print("Not a good deal! The sales are higher than $6M.")
else:
    print("Good deal! The sales are lower than $6M.")
```

✓ 0.0s

```
Expected Average Sales: $5.42M
Probability of sales greater than $6M: 35.32%
Probability of sales greater than $11M: 3.13%
Good deal! The sales are lower than $6M.
```

//For 1000 rounds

```
import numpy as np

#Rounds: 1000
results = []

for rnd in range(N_ROUNDS):
    prob_patent = np.random.randint(0, 2)
    base_sales = round(np.random.triangular(1e6, 3e6, 9e6) / 1e6, 2)
    patent_markup = np.random.triangular(0.25, 0.5, 0.75)
    final_sales = round(base_sales + (base_sales * prob_patent * patent_markup), 2)

    results.append(final_sales)

#Average Expected Sales
average_sales = np.mean(results)
#Probability of selling it more than $6M
prob_above_6M = np.sum(np.array(results) > 6) / N_ROUNDS
#Probability of selling it more than $11M
prob_above_11M = np.sum(np.array(results) > 11) / N_ROUNDS
#Results
print(f"Expected Average Sales: ${average_sales:.2f}M")
print(f"Probability of sales greater than $6M: {prob_above_6M:.2%}")
print(f"Probability of sales greater than $11M: {prob_above_11M:.2%}")
#Is it a good deal?
if average_sales > 6:
    print("Not a good deal! The sales are higher than $6M.")
else:
    print("Good deal! The sales are lower than $6M.")
```

✓ 0.0s

```
Expected Average Sales: $5.39M
Probability of sales greater than $6M: 36.00%
Probability of sales greater than $11M: 3.10%
Good deal! The sales are lower than $6M.
```

Question no. 02 (Bonus Assignment)

```
import numpy as np

def simulateVirusSpread(n_computers=20, p_spread=0.1, dailyRemoval=5, trials=100):
    totalDays = []
    totalInfectedCounts = []
    allInfectedProb = 0

    for _ in range(trials):
        infected = set([0])
        days = 0

        while infected:
            days += 1
            newInfected = set(infected)

            for comp in infected:
                for target in range(n_computers):
                    if target not in infected and np.random.rand() < p_spread:
                        newInfected.add(target)

            if len(newInfected) <= dailyRemoval:
                newInfected.clear()
            else:
                newInfected = set(np.random.choice(list(newInfected), size=len(newInfected) - dailyRemoval, replace=False))

            infected = newInfected

        totalDays.append(days)
        totalInfectedCounts.append(len(infected))

        if len(infected) == n_computers:
            allInfectedProb += 1

    expectedTime = np.mean(totalDays)
    probabilityAllInfected = allInfectedProb / trials
    expectedNumInfected = np.mean(totalInfectedCounts)

    return expectedTime, probabilityAllInfected, expectedNumInfected

expectedTime, probabilityAllInfected, expectedNumInfected = simulateVirusSpread()
print(f"Expected time to remove the virus: {expectedTime:.2f} days")
print(f"Probability each computer gets infected at least once: {probabilityAllInfected:.4f}")
print(f"Expected number of computers that get infected: {expectedNumInfected:.2f}")
```

```
Expected time to remove the virus: 1.04 days
Probability each computer gets infected at least once: 0.0000
Expected number of computers that get infected: 0.00
```